Sprawozdanie RSAD#2 x.2020

1. **Opisz znaczenie poszczególnych bitów rejestrów UCSRA, UCSRB, UCSRC.**

USART Control and Status Register A-

Bit	7	6	5	4	3	2	1	0	
	RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM	UCSRA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

•Bit 7 RXC-RECEIVE COMPLETE:
Zakończono odbiór znaku
RCX==0 →BUFOR ODBIORU JEST PUSTY

•Bit 6 TXC-TRANSMIT COMPLETE:
Zakończono transmisję całej ramki
TXC==1→BUFOR TRANSMISJI JEST PUSTY

•Bit 5 UDRE- DATA REGISTER EMPTY:
informuje o tym, czy rejestr danych UDR jest pusty i możemy wpisać nową daną do wysłania (jest to sprzętowe rozwiązanie portu szeregowego)- dzięki temu nie nadpiszemy danych podczas wysyłania
UDRE==1 → BUFOR JEST PUSTY

•Bit 4 FE- FRAME ERROR:
Kontrola błędów ramki
FE==1 →kolejny znak w buforze odbioru został odebrany z błędem

•Bit 3 DOR-DATA OVER RUN
DOR==1 →przepełnienie ramki

•Bit 2 PE-PARITY ERROR:
Kontrola Parzystości

•Bit 1 U2X- DOUBLE THE USART TRANSMISSION SPEED
Dwukrotne zwiększenie prędkości transmisji

•Bit 0 MPCM- MULTI-PROCESSOR COMMUNICATION MODE
Wieloprocesorowa komunikacja

USART Control and Status Register B-

Bit	7	6	5	4	3	2	1	0	
	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	UCSRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

•Bit 7 RXCIE- RX COMPLETE INTERRUPT ENABLE
•Bit 6 TXCIE- TX COMPLETE INTERRUPT ENABLE
•Bit 5 UDRIE- DATA REGISTER EMPTY INTERRUPT ENABLE
powyższe bity dotyczą przerwań

•Bit 4 RXEN- RECEIVER ENABLE
•Bit 3 TXEN- TRANSMITER ENABLE
on/off odbiornika, domyślnie wyłączone
on/off nadajnika, domyślnie wyłączone

Linie nadajnika i odbiornika są na porcie PB0 i PB1- normalnie te linie funkcjonują jako zwykłe linie wejściowo-wyjściowe. Jeśli chcemy uruchomić nadajnik musimy aktywować powyższe bity (ustawić 1).

•Bit 2 UCSZ2- CHARAKTER SIZE
Razem z pozostałymi bitami UCSZ1 i UCSZ0 z rejestru UCSRC będzie odpowiadał za liczbę bitów
•Bit 1 RXB8- RECEIVER CATA BIT8
•Bit 0 TXB8-TRANSMITER DATA BIT 8
Powyższe 2 bity wykorzystywane są przy transmisji 9 bitowej w ramce

USART Control and Status Register C-

Bit	7	6	5	4	3	2	1	0	
	URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL	UCSRC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	0	0	0	0	1	1	0	

•Bit 7 URSEL - REGISTER SELECT
Ten bit związany jest, że pod tym samym adresem umieszczono dwa rejestry-UCSRC i UBRRH. Zastosowano takie rozwiązanie prawdopodobnie z braku przestrzeni adresowej. Wybór, do którego rejestru chcemy się odwołać odbywa się właśnie przez ten bit nr 7

•Bit 6 UMSEL - MODE SELECT
Ustala tryb transmisji. Domyślnie UMSEL==0 →transmisja asynchroniczna

•Bit 5:4 UPM1:0 PARITY MODE
Bity kontroli parzystości. Domyślnie UPM1:0==0 → wyłączona kontrola parzystości

•Bit 3 USBS - STOP BIT SELECT
Ustala liczbę bitów stopu. Domyślnie USBS==0 → 1 BIT STOPU

•Bit 2 :1 UCSZ1:0 CHARAKTER SIZE
Razem z UCSZ2 w rejestrze UCSRB. Możemy wysyłać od 5 do 9 bitów w ramce.
Domyślne dla 8 bitów: UCSZ2==0 UCSZ1==1 UCSZ0==1

•Bit 0 UCPOL CLOCK POLARITY
Bit związany z transmisją synchroniczną(polaryzacją zegara).

2.Opisz znaczenie podstawowych prefiksów protokołu NMEA 0183- protokół transmisji danych odbiornika GPS.
Prefiksy określa typ urządzenia -identyfikator źródła; w podanym przykładzie jest to GP czyli GPS, inne przykłady:
„GL" - GLONASS,
„GA" - Galileo,
„GN" - GLONASS + GPS itp.

Przykładowe dane generowane przez odbiornik GPS:

Ciąg RMC (Recommended minimum of data- Określane przez standard NMEA minimum danych, które musi wysyłać KAŻDY odbiornik):

$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A

- RMC – nagłówek,
- 123519 – Aktualność danych - 12:35:19 UTC,
- A – status (A – aktywny; V – nieaktywny),
- 4807.038,N – szerokość geograficzna (latitude) - 48 deg 07.038' N,
- 01131.000,E – długość geograficzna (longitude) - 11 deg 31.000' E,
- 022.4 – prędkość obiektu (liczona w węzłach),
- 084.4 – kąt śledzenia/poruszania się obiektu (w stopniach) – przydatny w celu określenia kierunku poruszania się obiektu, jeżeli urządzenie GPS nie jest wyposażone w kompas,
- 230394 – data (23 marca 1994),
- 003.1,W – odchylenie magnetyczne ziemi,
- *6A – suma kontrolna.

Ciąg GGA (Fix information- Dostarcza informacji na temat aktualnego położenia):

$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

- GGA – Identyfikator nagłówka,
- 123519 – Aktualność danych - 12:35:19 UTC,
- 4807.038,N – szerokość geograficzna (latitude) - 48 st. 07.038' N,
- 01131.000,E – długość geograficzna (longitude) - 11 st. 31.000' E,
- 1 – jakość pomiaru (opisana dalej),
- 08 – ilość śledzonych satelitów,
- 0.9 – horyzontalna dokładność pozycji (HDOP) (opisana dalej),
- 545.4,M – wysokość w metrach nad poziom morza,
- 46.9,M – wysokość geoid (powyżej elipsoidy WGS84),
- (puste pole) – czas od czasu ostatniego uaktualnienia DGPS,
- (puste pole) – numer ID stacji DGPS,
- *47 – suma kontrolna

Ciąg GSA (Overall Satellite data-Dostarcza informacji o satelitach):

$GPGSA,A,3,04,05,,09,12,,,24,,,,,2.5,1.3,2.1*39

- GSA – nagłówek,
- A – automatyczny wybór pozycji (2D lub 3D) /M – manualny/,
- 3 – pozycja 3D. Możliwe wartości to:
- 1 – brak ustalonej pozycji z tej satelity,
- 2 – pozycja 2D,
- 3 – pozycja 3D,
- 04,05... – Numery satelitów wykorzystane do wyznaczenia pozycji (miejsce dla 12 satelitów),
- 2.5 – DOP (dilution of precision) – precyzja wyznaczonej pozycji,
- 1.3 – HDOP (horizontal dilution of precision) – precyzja horyzontalna,
- 2.1 – VDOP (vertical dilution of precision) – precyzja wertykalna,
- *39 – suma kontrolna.

Sekwencja GLL (Lat/Lon data-Określa aktualną pozycję (długość i szerokość geograficzną)

$GPGLL,4916.45,N,12311.12,W,225444,A,*31

- GLL – nagłówek,
- 4916.45,N - szerokość geograficzna (latitude) – 49 st. 16.45 min N,
- 12311.12,W – długość geograficzna (longitude) – 123 st. 11.12 min W,
- 225444 – godzina ustalenia pozycji (22:54:44),
- A – status (A – aktywny; V – nieaktywny),
- *31 – suma kontrolna.

3. Kod programu z zajęć:

Części omawiane w poprzednim sprawozdaniu nie zostały poniżej komentowane.

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp 0x0030
.org 0x0030

START:
ldi r16,LOW(RAMEND)
out SPL, r16
ldi r17,HIGH(RAMEND)
out SPH, r17

ldi r16, 0b00000001
out DDRB, r16//ustawia jedną linie PB0 PORTU B jako wyjście

ldi r16, 0b11111111
out DDRA, r16//dodatkowy port A jako wyjście-pomocniczo na
diodach bedziemy prezentowac wysylane dane

// ustawienie parametrów UART
; zapisujemy rejestr UBRR
ldi r16, 25; dla 4MHz 9600-> UBRR= (4MHz/(16*9600))-1
out UBRRL, r16//rejestr składa się z części LOW i HIGH- dla
wartości 25 wystarczy zapisać dolny rejestr
ldi r16, 0
out UBRRH, r16


; uaktywnienie nadajnika I ODBIORNIKA (Enable TX&RX)
ldi r16, (1<<TXEN )|(1<<RXEN)/* 1 zostaje przesunieta w lewo
o 'TXEN'(4) pozycji */
out UCSRB,r16


; format ramki (Set frame format): 8data, 1stop bit
ldi r16,
(1<<URSEL)|(0<<USBS)|(1<<UCSZ1)|(1<<UCSZ0)/*ustawiamy przesył
8 bitów danych oraz jeden bit stopu*/
out UCSRC,r16
//-----------------------------------------------------//
```

```
MAIN:
/*jeśli sam nadajnik
ldi r16, 'R'
call USART_Transmit
out PORTA,r16
CALL wait_1s

ldi r16, 'S'
call USART_Transmit
out PORTA,r16
CALL wait_1s

ldi r16, 'A'
call USART_Transmit
out PORTA,r16
CALL wait_1s

ldi r16, 'D'
call USART_Transmit
out PORTA,r16
CALL wait_1s*/

/*nadajnik i odbiornik>>>>>>>>>>>>>>>>>>>>>>*/
//odbierzemy dane:
call USART_Receive
out PORTA, r16
//odebrane dane wysyłamy z powrotem:
;call USART_Transmit
call send_ASCII
;call wait_1s


jmp MAIN

jmp PC
/*---koniec głównego programu-----
podprogramy:-----------------------------------------------*/

.org 0x0100

send_0://na PB0 wysyłamy 0
cbi PORTB, 0//ustawia 0 na wybranej linii portu I/O (na bicie
nr.0)
call wait_104us//symbol musi trwac 104us
ret
//-------------------------------------------------------------
```

```
send_1://na PB0 wysyłamy 1
sbi PORTB, 0//ustawia 1 na wybranej linii portu I/O (na bicie
nr.0)
call wait_104us
ret
//------------------------------------------------------------
wait_104us:
push r16 //wsadza na stos zeby nie nadpisac
ldi r16, 129
Loop_Count:
dec r16
brne Loop_Count
pop r16//przywracamy pierwotną wartość r16
ret
//------------------------------------------------------------

test_C://testuje czy dany bit jest 0 czy 1
ror r16//przesuwamy najmłodszy bit rejestru r16 do rejestru C
brcc ZERO /*jeśli C jest 0 skacze do etykiety ZERO, jeśli C
jest 1 to pomija instrukcję i  wykonuje kolejną poniższą*/
call send_1
ret
//------------------------------------------------------------
ZERO:
call send_0
ret

//------------------------------------------------------------
wait_1s:
ldi  r18, 41
ldi  r19, 150
ldi  r20, 128
L1: dec  r20
brne L1
dec  r19
brne L1
dec  r18
brne L1
ret
//------------------------------------------------------------
```

```
send_ASCII:

//ramka-wysyła najpierw bit startu (0):
call send_0
//wysyla na port B 8 bitów danych
call test_C//przechodzimy do wysłania pierwszego bitu
call test_C
call test_C
call test_C
call test_C
call test_C
call test_C
call test_C
//bit stopu-koniec transmisji(1):
call send_1

RET


// ---------------------------------------------
// podprogramy sprzętowego nadajnika i odbiornika

USART_Transmit://nadawanie
; Wait for empty transmit buffer
sbis UCSRA,UDRE //jeśli UDRE jest 1 to pracuje w petli, jesli
nie jest spelniony warunek to wychodzimy z petli
rjmp USART_Transmit
; Put data (r16) into buffer, sends the data
; ldi r16, 0x31
out UDR,r16
ret

// -----------

USART_Receive://odbiór
; Wait for data to be received
sbis UCSRA, RXC
rjmp USART_Receive//instrukcja rjump jest krótsza
; Get and return received data from buffer
in r16, UDR
ret
```

Zadanie dom 1: Napisz program ECHO, który uaktywni sprzętowy odbiornik i nadajnik modułu UART dla parametrów 57600,8,N,1. Program czeka na odebranie znaku z terminala i natychmiast odsyła go przez port szeregowy. Zaprezentuj odebrany bajt na porcie A. Częstotliwość zegara procesora 8 MHz.

$UBRR = \frac{8*10^6}{16*57600} - 1 = 7.680(5) \approx 8$

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp 0x0030
.org 0x0030
START:
//ustawienie stosu
ldi r16,LOW(RAMEND)
out SPL, r16
ldi r17,HIGH(RAMEND)
out SPH, r17

ldi r16, 0b00000001
out DDRB, r16

ldi r16, 0b11111111
out DDRA, r16

// ustawienie parametrów UART
; zapisujemy rejestr UBRR
ldi r16, 8; dla 8MHz 57600
out UBRRL, r16
ldi r16, 0
out UBRRH, r16


; uaktywnienie nadajnika (Enable transmitter)
ldi r16, (1<<TXEN )|(1<<RXEN)// 1 zostaje przesuniety w lewo
o tyle pozycji TXEN
out UCSRB,r16

; format ramki
ldi r16, (1<<URSEL)|(0<<USBS)|(1<<UCSZ1)|(1<<UCSZ0)
out UCSRC,r16
```

```
MAIN:

/*nadajnik i odbiornik>>>>>>>>>>>>>>>>>>>>>>*/
//odbierzemy dane:
call USART_Receive
out PORTA, r16

//odebrane dane wysyłamy z powrotem:
call USART_Transmit
call wait_1s



jmp MAIN


jmp PC
//---koniec głównego programu---
// --- podprogramy-----------------------------------------
.org 0x0100
wait_1s:
ldi  r18, 41
ldi  r19, 150
ldi  r20, 128
L1: dec  r20
brne L1
dec  r19
brne L1
dec  r18
brne L1
ret
// ---------------------------------------------
// podprogramy sprzętowego nadajnika i odbiornika
USART_Transmit:
sbis UCSRA,UDRE
rjmp USART_Transmit
out UDR,r16
ret

// ---------------------------------------------

USART_Receive:
sbis UCSRA, RXC
rjmp USART_Receive
in r16, UDR
ret
```

Zadanie dom 2: Uzupełnij realizowany na zajęciach program nadawczo/odbiorczy o fragment obsługi przerwania od odbiornik portu szeregowego. (Uaktywnij odpowiednie przerwanie i przy obsłudze przerwania wywołaj podprogram odbierania i wysyłania znaku).

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp START

.org 0x0016//adresy gdzie ma skonczyc jak dostanie przerwanie
rjmp USART_Transmit
.org 0x0018
rjmp USART_Receive


.org 0x0030

START:
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r17, HIGH(RAMEND)
out SPH, r17

//Ustawienie portu A jako wyjście:
ldi r16, 0xFF
out DDRA, r16

//Ustawienie parametrów UART:
ldi r16, 8
out UBRRL, r16
ldi r16, 0
out UBRRH, r16

//Uaktywnienie nadajnika (Enable transmitter)
ldi r16, (1<<TXEN) | (1<<RXEN)
out UCSRB,r16
//Format ramki (Set frame format):
ldi r16, (1<<URSEL) | (0<<USBS) | (1<<UCSZ1) | (1<<UCSZ0)
out UCSRC, r16

sei //włącza obsługę przerwań

jmp MAIN
```

```
MAIN:
call wait_1s

jmp MAIN

.org 0x0100

USART_Transmit:
sbis UCSRA, UDRE
rjmp USART_Transmit
out UDR, r16
reti //powrót z przerwania

USART_Receive:
sbis UCSRA, RXC
rjmp USART_Receive
in r16, UDR
out PORTA, r16
reti

wait_1s:
ldi r18, 41
ldi r16, 150
ldi r20, 128
Loop:
dec r20
brne Loop
dec r19
brne Loop
dec r18
brne Loop

ret
```

Zadanie 3: Zapoznaj się z dodatkowymi możliwościami sterowania znakami wyświetlanymi na typowym terminalu (poszukaj informacji o: ANSII ESC code). Napisz program, który co 1 sek. na terminalu zaprezentuje twoje imię i nazwisko w wybranym kolorze (np. zielonym) w postaci:
Imię Nazwisko
Imię Nazwisko
Imię Nazwisko ... itd
Zastosuj również znaki sterujące przejściem do następnej linii i powrotem na początek nowej linii.

```
.nolist
.include "m16def.inc"
.list
.listmac
.device ATmega16
.cseg
.org 0x0000
jmp 0x0030
.org 0x0030

START:
ldi r16,LOW(RAMEND)
out SPL, r16
ldi r17,HIGH(RAMEND)
out SPH, r17

ldi r16, 0b00000001
out DDRB, r16
ldi r16, 0b11111111
out DDRA, r16

// ustawienie parametrów UART
ldi r16, 25; dla 4MHz 9600
out UBRRL, r16
ldi r16, 0
out UBRRH, r16

//uaktywnienie nadajnika
ldi r16, (1<<TXEN )|(1<<RXEN)
out UCSRB,r16


//format ramki
ldi r16, (1<<URSEL)|(0<<USBS)|(1<<UCSZ1)|(1<<UCSZ0)
out UCSRC,r16
```

```
MAIN:

//zielony- przed tekstem wyslac sekwencje znakow  \e[32m
ldi r16, 92    //   \
call USART_Transmit
ldi r16, 101   //   e
call USART_Transmit
ldi r16, 91    //   [
call USART_Transmit
ldi r16, 51    //   3
call USART_Transmit
ldi r16, 50    //   2
call USART_Transmit
ldi r16, 109   //   m
call USART_Transmit

//tekst:

ldi r16, 'E'
call USART_Transmit
```

```
ldi r16, 0x0A//nowa linia
call USART_Transmit

ldi r16, 0x0D//poczatek wiersza
call USART_Transmit

CALL wait_1s//odczekanie 1s









jmp MAIN//pętla nieskonczona


jmp PC
//---koniec głównego programu---
//--podprogramy----------------------------------------------
.org 0x0100

wait_1s:
ldi  r18, 41
ldi  r19, 150
ldi  r20, 128
L1: dec  r20
brne L1
dec  r19
brne L1
dec  r18
brne L1
ret
//-----------------------------------------------------------
// podprogramy sprzętowego nadajnika i odbiornika
USART_Transmit:
sbis UCSRA,UDRE
rjmp USART_Transmit
out UDR,r16
ret

// -----------------------------------------------------
```